

Advanced Optimization

Chapter 8 — Calculating Derivatives

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 8

Where do derivatives come from?

Every method so far uses ∇f ; Newton-type methods also use $\nabla^2 f$ or $\nabla^2 f p$. Coding them by hand is tedious and error-prone. Three ways to get them automatically:

- ▶ **Finite differencing** (§8.1): perturb x , watch f change. Based on Taylor's theorem. *Approximate.*
- ▶ **Automatic differentiation** (§8.2): apply the chain rule to the elementary operations in the code for f . *Exact up to roundoff.*
- ▶ **Symbolic differentiation**: Mathematica, Maple, Macsyma. Not treated here.

Derivatives also serve post-optimal **sensitivity analysis**: how does the solution move when the data change?

The forward-difference formula

Forward difference

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + \epsilon e_i) - f(x)}{\epsilon}. \quad (8.1)$$

Full gradient: f at x and at $x + \epsilon e_i$, $i = 1, \dots, n$: **$n + 1$ evaluations.**

Taylor (2.6), for some $t \in (0, 1)$, with $\|\nabla^2 f(\cdot)\| \leq L$ in the region of interest:

$$f(x + p) = f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x + tp) p, \quad (8.2)$$

$$|f(x + p) - f(x) - \nabla f(x)^T p| \leq (L/2) \|p\|^2. \quad (8.3)$$

Truncation error and roundoff error

With $p = \epsilon e_i$ in (8.3):

$$\frac{\partial f}{\partial x_i}(x) = \frac{f(x + \epsilon e_i) - f(x)}{\epsilon} + \delta_\epsilon, \quad |\delta_\epsilon| \leq (L/2) \epsilon. \quad (8.4)$$

So $\epsilon \rightarrow 0$? No: f is computed in floating point. Unit roundoff $u \approx 1.1 \times 10^{-16}$ (double).

If $|\text{comp}(f(\cdot)) - f(\cdot)| \leq uL_f$, with L_f a bound on $|f|$, the total error is

$$\underbrace{(L/2)\epsilon}_{\text{truncation}} + \underbrace{2uL_f/\epsilon}_{\text{cancellation}}. \quad (8.5)$$

Read: pp. 194–196.

The best ϵ

Minimizing (8.5) over ϵ :

$$\epsilon^2 = \frac{4L_f u}{L}.$$

If the problem is well scaled, L_f/L is of modest size, and a near-optimal choice is

$$\epsilon = \sqrt{u} \approx 10^{-8}. \quad (8.6)$$

The total error is then about $\sqrt{u}$: **only about half the digits are correct.**

- ▶ This value is the default in many optimization packages.
- ▶ Good enough for many line searches and quasi-Newton methods.

The central-difference formula

Central difference

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon}. \quad (8.7)$$

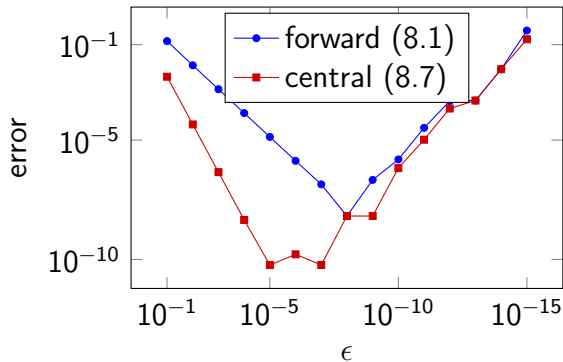
If $\nabla^2 f$ is Lipschitz continuous,

$$f(x + p) = f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x) p + O(\|p\|^3). \quad (8.8)$$

Set $p = \pm \epsilon e_i$ and subtract: the ϵ^2 terms cancel, and the error is $O(\epsilon^2)$.

With roundoff: optimal $\epsilon \approx u^{1/3}$, error $\approx u^{2/3}$ (Exercise 8.1). **Cost: $2n + 1$ evaluations**, about twice the forward difference.

Numerical example: the V-shaped error curve



$f(x) = e^x$ at $x = 1$, double precision. (Not in the book.)

- ▶ Right of the minimum: truncation, slope 1 and slope 2.
- ▶ Left of the minimum: cancellation, slope -1 .
- ▶ Forward: best at $\epsilon \approx 10^{-8} = \sqrt{u}$, error 7×10^{-9} .
- ▶ Central: best at $\epsilon \approx 10^{-5} \approx u^{1/3}$, error 6×10^{-11} .

The Jacobian of a vector function

For $r : \mathbb{R}^n \rightarrow \mathbb{R}^m$,

$$J(x) = \left[\frac{\partial r_j}{\partial x_i} \right]_{\substack{j=1,\dots,m \\ i=1,\dots,n}} = \begin{bmatrix} \nabla r_1(x)^T \\ \vdots \\ \nabla r_m(x)^T \end{bmatrix}. \quad (8.9)$$

If J is Lipschitz with constant L , then $\|r(x+p) - r(x) - J(x)p\| \leq (L/2)\|p\|^2$ (8.10), so

$$J(x)p \approx \frac{r(x + \epsilon p) - r(x)}{\epsilon}, \quad \frac{\partial r}{\partial x_i}(x) \approx \frac{r(x + \epsilon e_i) - r(x)}{\epsilon}. \quad (8.11, 8.12)$$

- ▶ $J(x)p$: **one extra evaluation**, accurate to $O(\epsilon)$ (inexact Newton, §11.1).
- ▶ Full J one column at a time: $n + 1$ evaluations.

Example: a tridiagonal Jacobian

For $n = 6$:

$$r(x) = \begin{bmatrix} 2(x_2^3 - x_1^2) \\ 3(x_2^3 - x_1^2) + 2(x_3^3 - x_2^2) \\ 3(x_3^3 - x_2^2) + 2(x_4^3 - x_3^2) \\ \vdots \\ 3(x_n^3 - x_{n-1}^2) \end{bmatrix} \quad (8.13)$$

$$J = \begin{bmatrix} \times & \times & & & & \\ \times & \times & \times & & & \\ & \times & \times & \times & & \\ & & \times & \times & \times & \\ & & & \times & \times & \times \\ & & & & \times & \times \end{bmatrix} \quad (8.14)$$

Each r_j depends on only two or three x_i . A perturbation of x_1 changes only r_1, r_2 ; re-evaluating $r_3, \dots, r_6$ for it is wasted work.

Two columns from one evaluation

Perturb x_1 and x_4 together: $p = \epsilon(e_1 + e_4)$. Columns 1 and 4 share no row, so

$$[r(x + p)]_{1,2} = [r(x + \epsilon e_1)]_{1,2}, \quad [r(x + p)]_{3,4,5} = [r(x + \epsilon e_4)]_{3,4,5}. \quad (8.15)$$

Hence (8.16)–(8.18):

$$[J(x)e_1]_{1,2} \approx \frac{[r(x + p) - r(x)]_{1,2}}{\epsilon}, \quad [J(x)e_4]_{3,4,5} \approx \frac{[r(x + p) - r(x)]_{3,4,5}}{\epsilon}.$$

Likewise $p = \epsilon(e_2 + e_5)$ and $p = \epsilon(e_3 + e_6)$. **Three extra evaluations of r for every n :**

$$p = \epsilon(e_1 + e_4 + e_7 + \cdots), \quad p = \epsilon(e_2 + e_5 + e_8 + \cdots), \quad p = \epsilon(e_3 + e_6 + e_9 + \cdots).$$

Read: pp. 197–200.

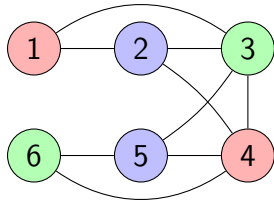
Graph coloring

Column intersection graph: nodes $1, \dots, n$; an arc $i-k$ if some r_j depends on both x_i and x_k .

Rule: connected nodes get different colors. One perturbation per color:

$$p = \epsilon(e_{i_1} + e_{i_2} + \dots + e_{i_\ell}).$$

- ▶ The minimum coloring is hard to find, but cheap heuristics do well (Curtis–Powell–Reid; Coleman–Moré).
- ▶ Newsam–Ramsdell: at most n_z extra evaluations, $n_z = \max$ nonzeros per row of J .



Graph for (8.13), $n = 6$; 3 colors (cf. Figure 8.1).

Approximating the Hessian

Apply the Jacobian techniques to $r = \nabla f$; symmetrize with $\frac{1}{2}(A + A^T)$.

Hessian–vector product (Newton–CG, Chapter 7):

$$\nabla f(x + \epsilon p) = \nabla f(x) + \epsilon \nabla^2 f(x) p + O(\epsilon^2), \quad (8.19)$$

$$\nabla^2 f(x) p \approx \frac{\nabla f(x + \epsilon p) - \nabla f(x)}{\epsilon}. \quad (8.20)$$

One gradient evaluation, error $O(\epsilon)$. Same as (7.10).

Function values only (from (8.8) with $p = \epsilon e_i, \epsilon e_j, \epsilon(e_i + e_j)$):

$$\frac{\partial^2 f}{\partial x_i \partial x_j}(x) = \frac{f(x + \epsilon e_i + \epsilon e_j) - f(x + \epsilon e_i) - f(x + \epsilon e_j) + f(x)}{\epsilon^2} + O(\epsilon). \quad (8.21)$$

Full Hessian: $n(n+1)/2 + n$ extra points.

Sparse Hessians: the arrowhead example

$$f(x) = x_1 \sum_{i=1}^n i^2 x_i^2. \quad (8.22)$$

Row 1 is full, so in the intersection graph for ∇f every node is connected to every other. Coloring needs n colors: **n extra gradients**.

For $n = 6$:

$$\nabla^2 f = \begin{bmatrix} \times & \times & \times & \times & \times & \times \\ \times & \times & & & & \\ \times & & \times & & & \\ \times & & & \times & & \\ \times & & & & \times & \\ \times & & & & & \times \end{bmatrix} \quad (8.23)$$

Exploiting symmetry

1. $p = \epsilon e_1$ gives column 1, hence row 1 too, by symmetry.
2. Only the diagonal entries $2, \dots, 6$ remain. They do not interfere:

$$p = \epsilon(e_2 + e_3 + \dots + e_6) = \epsilon(0, 1, 1, 1, 1, 1)^T, \quad (8.24)$$

$$\frac{\partial^2 f}{\partial x_i^2}(x) \approx \frac{\nabla f(x+p)_i - \nabla f(x)_i}{\epsilon}, \quad i = 2, \dots, 6.$$

Whole Hessian from ∇f at x and two other points, for any n .

Coloring rule for symmetric Hessians (Coleman–Moré)

Use the **adjacency graph** (arc $i-k$ if $i \neq k$ and $\partial^2 f / \partial x_i \partial x_k \neq 0$). Adjacent nodes get different colors, *and* every path of length 3 uses at least 3 colors.

Read: pp. 201–204.

Automatic differentiation

- ▶ Any function, however complicated, is evaluated as a sequence of **elementary operations** with one or two arguments: $+$, $-$, $\times$, $/$, a^b , $\sin$, $\exp$, $\log$, ...
- ▶ Apply the chain rule to each operation. For $h(y(x))$ with $y \in \mathbb{R}^m$:

$$\nabla_x h(y(x)) = \sum_{i=1}^m \frac{\partial h}{\partial y_i} \nabla y_i(x). \quad (8.25)$$

- ▶ The tool does the bookkeeping: source transformation (ADIFOR) or recording the computation at run time (ADOL-C).

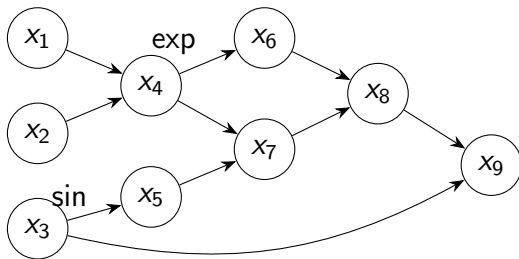
The results are **exact derivatives of the program**, up to roundoff. No ϵ , no truncation error.

Two basic modes: **forward** and **reverse**.

An example and its computational graph

$$f(x) = (x_1 x_2 \sin x_3 + e^{x_1 x_2}) / x_3 \quad (8.26)$$

$$\begin{aligned} x_4 &= x_1 x_2, & x_5 &= \sin x_3, \\ x_6 &= e^{x_4}, & x_7 &= x_4 x_5, & (8.27) \\ x_8 &= x_6 + x_7, & x_9 &= x_8 / x_3. \end{aligned}$$



Computational graph (cf. Figure 8.2)

Node i is a **parent** of node j if there is an arc $i \rightarrow j$. A node can be evaluated once all its parents are known: the **forward sweep**. The tool builds this graph itself; the user does not write (8.27).

The forward mode

Fix a **seed vector** $p \in \mathbb{R}^n$. Carry, alongside each x_i , its directional derivative

$$D_p x_i \stackrel{\text{def}}{=} (\nabla x_i)^T p = \sum_{j=1}^3 \frac{\partial x_i}{\partial x_j} p_j, \quad i = 1, \dots, 9. \quad (8.28)$$

Start: $D_p x_i = p_i$ for the independent variables. At each node, by the chain rule,

$$x_7 = x_4 x_5 : \quad D_p x_7 = x_5 D_p x_4 + x_4 D_p x_5. \quad (8.29)$$

At the end: $D_p x_9 = \nabla f(x)^T p$. **One sweep, one directional derivative.**

- The values $x_i, D_p x_i$ can be overwritten once all children of node i are done.

Forward mode at $x = (1, 2, \pi/2)^T$ with seed $p = e_1$

node	value x_i	rule	$D_{e_1} x_i$
x_1, x_2, x_3	1, 2, $\pi/2$	seed	1, 0, 0
$x_4 = x_1 x_2$	2	$x_2 D x_1 + x_1 D x_2$	2
$x_5 = \sin x_3$	1	$\cos x_3 D x_3$	0
$x_6 = e^{x_4}$	e^2	$e^{x_4} D x_4$	$2e^2$
$x_7 = x_4 x_5$	2	$x_5 D x_4 + x_4 D x_5$	2
$x_8 = x_6 + x_7$	$2 + e^2$	$D x_6 + D x_7$	$2 + 2e^2$
$x_9 = x_8 / x_3$	$(4 + 2e^2)/\pi$	$D x_8 / x_3 - x_8 D x_3 / x_3^2$	$(4 + 4e^2)/\pi$

So $\partial f / \partial x_1 = (4 + 4e^2)/\pi$. For $\partial f / \partial x_2$ and $\partial f / \partial x_3$: two more sweeps, with $p = e_2$ and $p = e_3$. (Exercise 8.7.)

Cost and implementation of the forward mode

Division $z \leftarrow w/y$ becomes

$$D_p z \leftarrow \frac{1}{y} D_p w - \frac{w}{y^2} D_p y. \quad (8.30)$$

- ▶ Full gradient: seeds $p = e_1, \dots, e_n$ carried together. One division in f costs about $2n$ multiplications and n additions: **work and storage grow by up to a factor n .**
- ▶ Sparse storage helps: many $D_{e_j} x_i$ are zero early in the graph.
- ▶ Implementation: a **precompiler** that writes the extended code, or **operator overloading** (C++), which extends the data types transparently.

Read: pp. 204–207.

The reverse mode

First the forward sweep computes f , and stores every x_i and every arc partial $\partial x_j / \partial x_i$. Then a **reverse sweep** goes from the output back to the inputs. Each node has an **adjoint** $\bar{x}_i$; start with $\bar{x}_i = 0$, except $\bar{x}_N = 1$ at the output.

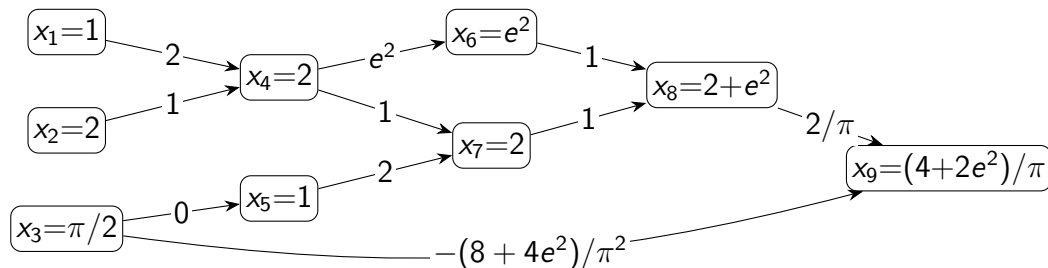
Reverse sweep

$$\frac{\partial f}{\partial x_i} = \sum_{j \text{ a child of } i} \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial x_i}, \quad \bar{x}_i += \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial x_i}. \quad (8.31, 8.32)$$

When all children of i have contributed, $\bar{x}_i = \partial f / \partial x_i$: node i is **finalized** and passes its contribution on to its parents.

The sweep works with **numbers, not formulas**.

Reverse mode at $x = (1, 2, \pi/2)^T$: the arc partials



Arc labels: $\partial x_j / \partial x_i$, computed and stored during the forward sweep (cf. Figure 8.3). Example: $x_9 = x_8 / x_3$ gives $\partial x_9 / \partial x_3 = -x_8 / x_3^2 = -(8 + 4e^2) / \pi^2$.

Reverse mode at $x = (1, 2, \pi/2)^T$: the sweep

Start with $\bar{x}_9 = 1$; node 9 is finalized. Its parents are nodes 3 and 8:

$$\bar{x}_3 += 1 \cdot \frac{-(2 + e^2)}{(\pi/2)^2} = \frac{-8 - 4e^2}{\pi^2}, \quad \bar{x}_8 += 1 \cdot \frac{1}{\pi/2} = \frac{2}{\pi}. \quad (8.33)$$

Node 3 still waits for node 5. Node 8 is finalized, and so on down the graph:

$$\begin{aligned} \bar{x}_6 = \bar{x}_7 = \frac{2}{\pi}, \quad \bar{x}_4 = \frac{2}{\pi}e^2 + \frac{2}{\pi} \cdot 1 = \frac{2+2e^2}{\pi}, \quad \bar{x}_5 = \frac{2}{\pi} \cdot 2 = \frac{4}{\pi}, \\ \bar{x}_3 += \frac{4}{\pi} \cdot 0, \quad \bar{x}_1 = \bar{x}_4 \cdot 2, \quad \bar{x}_2 = \bar{x}_4 \cdot 1. \end{aligned}$$

Result: one reverse sweep gives the whole gradient

$$\nabla f(x) = (\bar{x}_1, \bar{x}_2, \bar{x}_3) = \left(\frac{4+4e^2}{\pi}, \frac{2+2e^2}{\pi}, \frac{-8-4e^2}{\pi^2} \right).$$

The cheap gradient, and its price

Cost of the reverse mode for $f : \mathbb{R}^n \rightarrow \mathbb{R}$

The gradient costs **at most 4–5 times** the arithmetic of f alone, **independent of n** .

Example (8.33): two multiplications, a division and two additions, against one division in the forward sweep. The forward mode needs up to n times the work of f .

The price is memory. The whole graph must be stored for the reverse sweep. At 20 bytes per node, one second of a 100 Mflop computation can need 2 GB.

- ▶ **Checkpointing:** store only some states; recompute the pieces in between during the reverse sweep. This trades extra arithmetic for less storage.

Read: pp. 207–210.

Vector functions and partial separability

For $r : \mathbb{R}^n \rightarrow \mathbb{R}^m$ the graph has m output nodes. **Partially separable** f (Chapter 7):

$$f(x) = \sum_{i=1}^{ne} f_i(x), \quad r(x) = \begin{bmatrix} f_1(x) \\ \vdots \\ f_{ne}(x) \end{bmatrix}, \quad \nabla f(x) = J(x)^T e, \quad (8.34, 8.35)$$

with $e = (1, \dots, 1)^T$. Each f_i depends on a few x_j : J is sparse, and coloring makes it cheap.

- ▶ Gay: automatic detection of the partially separable structure.
- ▶ Constrained problems: evaluate f and all c_j together as one vector function; shared subexpressions (like x_4 in (8.27)) are computed once.

Jacobians: Jp by forward mode, $J^T q$ by reverse mode

Forward, seed $p \in \mathbb{R}^n$

One sweep gives $J(x)p$.

Full J : n seeds, or fewer with coloring of columns.

Cost $\approx$ number of seeds $\times$ cost of r .

Reverse, seed $q \in \mathbb{R}^m$

One sweep on $r(x)^T q$ gives

$$\nabla[r(x)^T q] = J(x)^T q.$$

Full J : m seeds, or coloring of J^T .

Cost $\leq 5 \times$ number of seeds $\times$ cost of r .

- ▶ $m \ll n$: reverse. $m \gg n$: forward. The two can be combined (some columns by forward, the remaining rows by reverse).
- ▶ Inexact Newton (§11.1) needs only $J(x)p$: **one forward sweep**, about the cost of r .

Read: pp. 210–213.

Hessians by the forward mode

For two seeds p, q , carry also

$$D_{pq}x_i = p^T(\nabla^2 x_i) q, \quad D_{pq}x_i = 0 \text{ at the inputs.} \quad (8.36)$$

Propagation rules, from the chain rule:

$$x_i = x_j + x_k : \quad D_p x_i = D_p x_j + D_p x_k, \quad D_{pq} x_i = D_{pq} x_j + D_{pq} x_k, \quad (8.37)$$

$$x_i = L(x_j) : \quad D_p x_i = L'(x_j) D_p x_j, \quad (8.38b)$$

$$D_{pq} x_i = L''(x_j)(D_p x_j)(D_q x_j) + L'(x_j) D_{pq} x_j. \quad (8.38c)$$

At the output: $D_{pq}f = p^T \nabla^2 f(x) q$. Note that (8.38c) needs both D_p and D_q .

Cost of forward-mode Hessians

- ▶ **Full Hessian:** pairs (e_j, e_k) , $k \leq j$, only where $[\nabla^2 f]_{jk}$ may be nonzero. Work: a small multiple of $1 + n + N_z(\nabla^2 f)$ times the cost of f .
- ▶ **Hessian–vector product** $\nabla^2 f(x)q$: carry $D_{e_j}x_i, D_q x_i, D_{e_j q} x_i$. The output holds $e_j^T \nabla^2 f(x)q = [\nabla^2 f(x)q]_j$. Work: a small multiple of $2n$.

Univariate trick. Avoid the cross terms D_{pq} , $p \neq q$:

$$[\nabla^2 f]_{ij} = \frac{1}{2} [(e_i + e_j)^T \nabla^2 f(e_i + e_j) - e_i^T \nabla^2 f e_i - e_j^T \nabla^2 f e_j]. \quad (8.39)$$

With $\psi(t) = f(x + tp)$ (8.40): $D_p f = \psi'(0)$ and $D_{pp} f = \psi''(0)$. This extends to higher derivatives.

Hessians by the reverse mode

Hessian–vector product: reverse over forward

1. Forward sweep: compute f and $\nabla f(x)^T q$ (carry x_i and $D_q x_i$).
2. Reverse sweep on the scalar $\nabla f(x)^T q$:

$$\frac{\partial}{\partial x_i} (\nabla f(x)^T q) = [\nabla^2 f(x) q]_i, \quad i = 1, \dots, n.$$

- ▶ Cost: about **12 times** the cost of f , **independent of n** .
- ▶ Full Hessian: $q = e_1, \dots, e_n$, up to $12n$. Sparse with known structure: $12N_c$, where N_c is the number of seed vectors from coloring.

Read: pp. 213–216.

Current limitations

- ▶ **AD differentiates the program, not the mathematics.** If f needs a PDE solve, the computed value is $\hat{f}(x) = f(x) + \tau(x)$. The truncation error $|\tau|$ is small, but τ' need not be, so $\hat{f}'$ can be far from f' . Finite differences have the same problem.
- ▶ **Branches.** The linear function $f(x) = x - 1$, coded as

```
if (x == 1.0) then f = 0.0 else f = x - 1.0
```

gives $f'(1) = 0$ by AD.

The book's conclusion

AD is a powerful set of tools, but not a panacea: the user is still responsible for thinking about derivative calculations.

Read: pp. 216–217.

AD today (not in the book)

- ▶ The reverse mode is **backpropagation**. Training a network with millions of parameters is feasible because the gradient costs a small constant times the loss, independent of n .
- ▶ JAX and PyTorch expose both modes: $\text{jvp} = \text{forward mode } (Jp)$, $\text{vjp} = \text{reverse mode } (J^T q)$. `jax.hessian` uses forward over reverse.
- ▶ Checkpointing is “gradient checkpointing” in deep learning: the same memory–compute trade-off.

Chapter 8 in one table

Quantity	Method	Cost (in units of f or r)
∇f , error $\sqrt{u}$	forward difference (8.1)	$n + 1$
∇f , error $u^{2/3}$	central difference (8.7)	$2n + 1$
∇f , exact	forward mode	$\sim n$
∇f , exact	reverse mode	≤ 5
$J(x)p$	FD (8.11) or forward mode	1
sparse J	FD or AD with coloring	number of colors
$\nabla^2 f q$	FD of gradient (8.20)	1 gradient
$\nabla^2 f q$	reverse over forward	~ 12

Three results to carry forward

1. **Finite differences trade truncation against cancellation.** Forward: $\epsilon = \sqrt{u}$, about half the digits. Central: $\epsilon = u^{1/3}$, about two thirds.
2. **Sparsity makes differencing cheap.** Columns that share no row are perturbed together; graph coloring picks the groups. Symmetry reduces the count further for Hessians.
3. **The reverse mode gives the gradient at a constant cost.** At most about 5 times the cost of f , for any n , paid for in memory.

Worked through: (8.26) in both modes, the arrowhead Hessian (8.22).

Homework (problems 5–6 are not in the book)

1. Exercise 8.1: for the central difference, $\epsilon \approx u^{1/3}$ and error $\approx u^{2/3}$.
2. Exercise 8.5: adjacency graph of (8.22); show that the two-color scheme is valid.
3. Exercise 8.9: finish the reverse sweep for (8.26) at $x = (1, 2, \pi/2)^T$; give the order in which nodes are finalized.
4. Exercise 8.10: reverse-sweep operations for $x_k \leftarrow x_i x_j$ and $x_k \leftarrow \cos x_i$.
5. Reproduce the V-shaped plot for $f(x) = \sin x$ at $x = 1$, $\epsilon = 10^{-1}, \dots, 10^{-15}$. Mark $\sqrt{u}$ and $u^{1/3}$.
6. Write a gradient checker: compare your analytic (or AD) gradient of Rosenbrock's function (2.22) with central differences, and report the largest relative error.